

RP-04 — Build Tooling & GitHub Pages Deployment: Group Lesson Planner

Metadata

Field	Value
Report id	RP-04
Date	2026-08-20
Subject	Moving <code>https://denyslystopadskyy.github.io/lesson-planner/</code> from one unbundled HTML file to a bundled React application with the live URL unchanged
Inputs consumed	RP-01 (<code>docs/research/rp01-app-inventory/rp01-app-inventory.md</code> , read in full); the research agent contract; committed <code>AGENTS.md</code> and <code>.gitignore</code> ; committed <code>index.html</code> (grep only, not modified); uncommitted prior art in the main checkout (<code>package.json</code> , <code>playwright.config.ts</code> , <code>tsconfig.json</code>)
Locally measured	Live response headers and status codes for <code>/lesson-planner/</code> , <code>/assets/index.js</code> , <code>/</code> and a nonexistent path on <code>denyslystopadskyy.github.io</code> ; local <code>node -v 24.10.0</code> and <code>npm -v 11.6.0</code> ; <code>npm registry dist-tags</code> for 11 packages; the <code>create-vite template-react-ts</code> config files byte-for-byte
Externally sourced	Vite, React, TypeScript, ESLint, typescript-eslint, Biome, Oxc, Playwright, GitHub Pages and GitHub Actions official documentation; the five Pages action repositories' own <code>action.yml</code> and Releases API
Consumed from the orchestrator addendum	The GitHub Pages publishing source is " Deploy from a branch ", <code>main / (root)</code> — RP-01's TBD, resolved by public-evidence inference on 2026-08-20 [S46]. Also: every npm script in <code>AGENTS.md</code> is aspirational, since no root <code>package.json</code> is committed
Not verified	The end user's browser and operating system; the built bundle's wire weight (nothing has been built yet); the exact failure mode of running a Pages workflow while the source is still branch-based
Verification statement	Every version number, default value, config key, action tag and quota in this report was read from a primary source on 2026-08-20 and is cited. No build was executed and no repository file was modified, so every claim about the <i>proposed</i> configuration is design intent, labelled as such, not a measurement. The four live-URL measurements are my own <code>curl</code> observations on 2026-08-20.

Executive summary

The recommended stack is **Vite 8 + React 19 + TypeScript 6.0**, deployed by a **GitHub Actions workflow that uploads a Pages artifact**, with **no client-side router**. Create React App was formally deprecated by the React team on 2025-02-14 [S7], so it is not a candidate.

Four decisions carry the report:

1. **base must be `'/lesson-planner/'`** — leading and trailing slash — in `vite.config.ts`. This is the single most consequential line in the whole migration. Vite's default is `'/'` [S1], which emits absolute asset URLs resolving to the domain root. I measured `https://denyslystopadskyy.github.io/assets/index.js` returning **404** today, so the failure is exactly the classic one: a blank page with 404s on every asset [S43].
2. **No router.** RP-01 established zero use of hash, query or `history` in the deployed app, one screen and three modals. Worse for routing: group identity is an **array index** (RP-01 §3), so a deep link would be unstable by construction. Modal state belongs in React state. Hash-vs-history is a contingency, not a decision for now.
3. **Ship the React build to a staging subpath on the same origin first** — `https://denyslystopadskyy.github.io/lesson-planner/next/` — while the current page keeps `/lesson-planner/` byte-for-byte. This is the only continuity option that preserves the end user's data, and the same property makes it dangerous: same origin means the staging build can corrupt the live data. The plan therefore pairs it with a key-prefix isolation flag and an export-first rule.
4. **Cutover is one environment variable in one workflow file**, and rollback is reverting that one line. Both are numbered and executable in §6.

One prerequisite sits outside the code and gates everything else. The site is published today by **"Deploy from a branch", source `main / (root)`** [S46], which is why it works with no workflow at all. An artifact-based workflow requires the owner to select **GitHub Actions** in Settings → Pages [S8] — a manual, one-time, web-UI action that no script here can perform. §4 treats it as step 0, and §9 R3 records it as the plan's single point of failure. At that same screen, **never** choose the `main / /docs` option: `docs/research/` now holds the research reports, and that choice would publish them in place of the application.

The origin-and-keys invariant

`localStorage` is scoped to an **origin**, not to a path [S41]. The origin is `https://denyslystopadskyy.github.io`. The keys are exactly three: `groupLessonPlannerData` (`index.html:377`), `groupLessonPlannerSettings` (`index.html:378`) and `paymentTemplate` (`index.html:1208` , `index.html:1211`) — the third stored as a raw string, not JSON.

Any change that alters the origin makes the teacher's data unreachable, and any build that reads different key names makes it invisible. There is no backup and she has already lost her data once. Every step in §5 and §6 is written to hold this invariant, and a grep of the built bundle for the three literal key names is a mandatory cutover gate.

A side effect worth naming: because `denyslystopadskyy.github.io` is **one origin for every GitHub Pages project under that account**, any other site published from that account shares this `localStorage` namespace and can read the teacher's data. Moving the app to a different repository does not isolate it.

1. Recommended stack with justification

Tool	Role	Alternative considered	Why chosen
Vite 8.2.2	Dev server, bundler, static build	Create React App; Parcel; Rspack; Next.js; plain esbuild	CRA is deprecated [S7]. Vite is the React team's first-listed build tool [S7], <code>npm create vite@latest</code> ships a first-party <code>react-ts</code> template [S6], and its own docs carry a GitHub Pages recipe [S2]. Its <code>preview</code> default port is 4173 [S4] — the same port <code>AGENTS.md:11</code> names and the uncommitted <code>playwright.config.ts</code> sets
React 19.2.8	UI layer	Preact; Lit; Svelte; staying vanilla	JSX escaping removes RP-01's stored-XSS class (D1) outright rather than patching it. <code>@vitejs/plugin-react</code> 6.1.0 peers <code>vite ^8.0.0</code> [S39], so the pairing is first-party. Preact would be smaller but the developer's ecosystem knowledge is React
TypeScript ~6.0.x	Type checking (<code>tsc -b</code> , no <code>emit</code>)	TypeScript 7.0.2; staying on JavaScript	<code>create-vite</code> 's own <code>react-ts</code> template pins <code>typescript: ~6.0.2</code> [S28]. TS 7.0 ships no stable API — "TypeScript 7.0 is no longer ship[ping] with an API" [S21] — and <code>typescript-eslint</code> caps at <code><6.1.0</code> [S24]. TS 6.0 also flips <code>strict</code> to true by default [S22], which satisfies <code>AGENTS.md</code> 's <code>no- any</code> rule with zero config
Playwright 1.62.1	End-to-end tests	Cypress; WebdriverIO	Already the developer's toolchain, already configured in the main checkout, and RP-01's runtime verification was done with it. Do not replace what already works
Vitest 4.1.11	Unit tests for pure domain functions	Jest; Node's built-in test runner	Peers <code>vite ^6 ^7 ^8</code> [S33], so it reuses the same config and transform pipeline — no second toolchain. Scoped to money arithmetic, month-key helpers and CSV parse/serialise, which is precisely where RP-01's D4, D12 and D20 live
Biome 2.5.9	Linter and formatter, one binary	ESLint 10 + <code>typescript-eslint</code> + Prettier; <code>oxlint</code> + <code>oxfmt</code>	One dev dependency, one config file, one command, stable 2.x. See §7 for the full comparison
GitHub Actions	Build, test, deploy	Branch-based publishing of committed build output	<code>AGENTS.md:36</code> forbids committing generated artifacts. Artifact deployment is the only option that never puts <code>dist</code> in git

Version-verified baseline, all checked 2026-08-20

Package	Latest	Source
<code>vite</code>	8.2.2 (published 2026-08-20)	[S30]

Package	Latest	Source
react , react-dom	19.2.8	[S31]
@vitejs/plugin-react	6.1.0, peers vite ^8.0.0	[S39]
typescript	7.0.2 latest, 6.0.3 the 6.x line	[S32]
@playwright/test	1.62.1	[S37]
vitest	4.1.11	[S33]
@biomejs/biome	2.5.9	[S34]
oxlint	1.79.0	[S35]
oxfmt	0.64.0 — pre-1.0	[S36]
eslint	10.8.1	[S38]
typescript-eslint	8.67.0, peers typescript >=4.8.4 <6.1.0	[S24]

Vite requires Node.js 20.19+ or 22.12+ [S6]. The local machine runs Node 24.10.0, which satisfies that.

The Create React App question, settled

React's official blog post *Sunsetting Create React App*, dated **2025-02-14**, states: "Today, we're deprecating Create React App for new apps" [S7]. It recommends frameworks first (Next.js, React Router, Expo) and build tools second (Vite, Parcel, RSBuidl). A framework is the wrong shape here: there is no server, no data fetching and no routing, so Next.js's entire value proposition is inapplicable and its static-export mode would add configuration for nothing. Vite is the correct branch of that recommendation.

One thing this migration makes worse, stated plainly

RP-01 §6 measured the current page at **58,649 bytes uncompressed, 13,610 bytes gzipped, in exactly one request**. A React build adds `react` and `react-dom` and splits into at least three requests (document, JS chunk, CSS chunk). The bundled app will be **heavier on the wire and slower to first paint than what is deployed today**. The exact figure is TBD — resolved by running `npx vite build` once and reading the reported chunk sizes, which cannot be done before the app exists. The justification for this migration is maintainability, testability and the removal of the XSS and unhandled-error defect classes; it is **not** performance, and it should not be sold as such.

2. GitHub Pages configuration requirements

Concern	Setting	Failure mode if wrong
Project-scoped subpath	base: '/lesson-planner/' in <code>vite.config.ts</code>	Default base: '/' [S1] emits <code>/assets/index-HASH.js</code> ; the browser requests <code>https://denyslystopadskyy.github.io/assets/...</code> which I measured as 404 [S43]. Result: white page, console 404s, no error message

Concern	Setting	Failure mode if wrong
Trailing slash	Write both slashes: <code>'/lesson-planner/'</code>	Vite's documented forms are <code>/foo/</code> and <code>https://bar.com/foo/</code> [S1]. <code>actions/configure-pages</code> emits <code>base_path</code> as <code>/my-repo</code> with no trailing slash [S15]; wiring that output straight into <code>--base</code> deviates from the documented form. Hardcode the literal instead
Relative base	<code>base: './'</code> is documented for "embedded deployment" [S1]	Works for one flat page and survives a path change, but breaks the moment any nested URL exists. Valid fallback, not the recommendation
Publishing source, today	"Deploy from a branch", <code>main / (root)</code> [S46]	Not a misconfiguration — it is why the app works today with no workflow. But it means the served site <i>is</i> the repository root, so any bundled build would have to be committed there
Publishing source, required	Settings → Pages → Build and deployment → GitHub Actions [S8]. A one-time change only the repository owner can make	Until it is flipped, a custom workflow's deploy step does not put content on the site. GitHub's docs do not state the exact error [S8][S9]; TBD, resolved by one trial run
The <code>docs/research/</code> tree	Leave the publishing source at root, then move it to GitHub Actions	Inert today. But switching to <code>main / /docs</code> instead of GitHub Actions would publish the research reports in place of the application [S8][S46]. Never pick the <code>/docs</code> option for this repository
Artifact root maps to site root	Upload the directory whose root holds the <code>index.html</code> you want at <code>/lesson-planner/</code>	Uploading a <code>dist</code> that contains only the React build silently replaces the live page. Inference from Vite's instruction to upload <code>dist</code> while setting <code>base</code> to <code>/<REP0>/</code> [S2] plus <code>deploy-pages</code> serving the uploaded tree [S13]
Artifact name	Leave <code>upload-pages-artifact</code> name at its default <code>github-pages</code> [S14]	<code>deploy-pages</code> looks for an artifact named <code>github-pages</code> [S13]; renaming one side without the other makes deploy hang then fail
Artifact path	<code>upload-pages-artifact</code> path defaults to <code>_site/</code> [S14] — must be set explicitly	Omitting <code>path</code> uploads nothing useful for a Vite build
Workflow permissions	<code>contents: read, pages: write, id-token: write</code> [S9] [S13]	<code>id-token: write</code> is required to mint the OIDC JWT that authenticates the deployment [S13]. Missing it fails the deploy job
Deployment environment	<code>environment: name: github-pages</code> [S9][S13]	Omitting the environment breaks the deployment protection contract <code>deploy-pages</code> relies on
Job dependency	<code>deploy</code> must declare <code>needs: build</code> [S9]	Without <code>needs</code> , the deploy job runs independently and searches for an artifact that does not exist yet [S9]

Concern	Setting	Failure mode if wrong
Jekyll and underscore files	Nothing needed. Vite emits <code>assets/</code> , not <code>_assets/</code> [S3]	A <code>.nojekyll</code> file would not even ship: <code>upload-pages-artifact</code> sets <code>include-hidden-files: false</code> by default [S14]. Whether Jekyll runs at all for an Actions-deployed artifact is TBD — resolved by publishing one underscore-prefixed file and checking whether it is served
Secure context	Keep <code>https://</code> on the deployed origin; use <code>http://localhost</code> locally	RP-01 established <code>navigator.clipboard</code> requires a secure context. Serving the dev build over a LAN IP on plain HTTP silently breaks Copy Payment Message
Cache staleness	Nothing to configure	I measured <code>cache-control: max-age=600</code> on the live response [S43]. Expect up to 10 minutes before a deployment is visible. Vite's content-hashed asset filenames prevent a stale-HTML-plus-new-asset mismatch, but the HTML itself can be served from cache
Custom domain	Do not add one during this migration	A custom domain changes the origin and orphans all three <code>localStorage</code> keys [S41]
Size and rate limits	Nothing to configure	Site limit 1 GB, soft bandwidth 100 GB/month, soft build limit 10 builds per hour , 10-minute deployment timeout [S10][S13]. Only the 10-builds-per-hour figure is reachable here, by pushing repeatedly during a debugging session
404 page	Optional <code>publish/404.html</code> [S42]	Not needed — see §3

The minimum viable `vite.config.ts`

The React application lives under `app/` so that Vite never treats the legacy repo-root `index.html` as its build entry. That collision is real: Vite's entry is `<root>/index.html`, and the legacy file must stay at the repo root until the publishing source has been switched (see §5).

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  root: 'app',
  // Project-scoped GitHub Pages site. Leading AND trailing slash are both required.
  // Overridden to '/lesson-planner/next/' for the pre-cutover staging build.
  base: '/lesson-planner/',
  plugins: [react()],
});
```

Because `base` is set, the dev server and `vite preview` serve the app **under `/lesson-planner/`, not at the server root** — *inference*, from `--base` being a documented flag on both `vite` and `vite preview` [S5] and from preview inheriting the shared options [S4]; verify with one `npm run dev` and read the URL

it prints. The practical consequence is firm either way: `playwright.config.ts`'s `baseURL` has to include the base path, which is why §4 serves the assembled tree with a plain static server rather than relying on preview's path handling.

3. Routing decision with recommendation

Recommendation: add no router. Not react-router, not wouter, not a hand-rolled one.

The evidence is unambiguous. RP-01 §5 records the routing-and-URL row as "None. Zero use of hash, query, `history` or `pushState`", and RP-01 §7 lists "zero URL or hash or history usage" among the app's isolation properties. The feature inventory in RP-01 §2 is one screen plus three modals, all four of which "live in the DOM simultaneously; none is ever created or destroyed". There are no deep links to preserve, no back-button semantics in use, and nothing a router would restore.

There is a second, stronger argument that RP-01 supplies and that outlives the current design: **group identity is the array index** (`index.html:604` , and CSV import re-keys by name). A URL such as `#/group/3` would point at whichever group happened to occupy slot 3, and would silently point somewhere else after any add, delete or import. Routing cannot be added meaningfully until entities acquire stable identifiers. Adding a router before that would encode a bug in the URL bar.

Modal open/closed state is React state — one discriminated union in the top-level component covers all three modals and removes RP-01's D8 and D9 (closed modals staying focusable) as a side effect, because a closed modal simply is not rendered.

Contingency, if deep links are ever wanted

This is a "later, if the data model gains stable ids" note, not work to schedule now.

- **Hash routing** (`#/group/abc123`) needs no server configuration, no 404 fallback and no build change. On static hosting it is the cheaper option and the one to reach for first.
- **History routing** requires a fallback because GitHub Pages has no rewrite rules: copy the built `index.html` to `404.html` inside the publish directory [S42]. Trade-offs: every mistyped path renders the whole application instead of a not-found page; the HTTP status stays 404 for real routes, so crawlers and any uptime monitor see failures on valid URLs; and the well-known redirect variant (storing the path, redirecting to the root, then replaying it with `history.replaceState`) adds a visible flash and writes an extra history entry.

Given the app is a single-user private tool with no crawling, no sharing and no incoming links, neither is worth the code today.

4. CI/CD workflow design, step by step

Step 0, and it is a prerequisite rather than a step: switch the publishing source

The site is served today by "**Deploy from a branch**", source `main / (root)` — orchestrator-verified on 2026-08-20 from the absence of any `gh-pages` branch, a live `last-modified` 42 seconds after commit `cf69aa4` on `main`, a `main` tree containing only `index.html`, `AGENTS.md`, `LICENSE` and `.gitignore`, no `.github/` directory anywhere, and served bytes identical to `main:index.html` [S46].

Everything in this section is inert until **Settings → Pages → Build and deployment → GitHub Actions** is selected [S8]. That is a repository-owner action in the web UI; no workflow, script or API call in this plan can perform it, and no amount of green CI substitutes for it. Treat it as an explicit gated step, sequenced as §5 phase 1 sequences it: prove one green run first, then flip.

Two things follow. First, **the `/docs` branch option must never be chosen for this repository** — `docs/research/` now holds the research reports, so selecting `main / /docs` would publish those instead of the application [S8][S46]. Second, if the owner declines to switch, the only branch-based route is committing built output to the root of `main`, which is rejected below.

Design decisions

- **Deploy from a workflow artifact, not a branch.** Branch publishing needs the built output committed — into `main` root (which is what the current source serves, so `dist` contents would land beside `index.html`), into `main /docs` (which would publish the research reports instead of the app), or onto a `gh-pages` branch that does not exist today [S46]. All three commit generated artifacts, which `AGENTS.md:36` forbids. Artifact deployment keeps git clean.
- **Two jobs, not three.** `build` does lint, type-check, unit tests, assemble, end-to-end tests and artifact upload. `deploy` does nothing but `deploy-pages`, gated by `needs: build`. One job means one `npm ci` and one Playwright browser download; splitting them would double both for no gain at this scale.
- **The tests gate the deploy structurally, not by convention.** The `upload-pages-artifact` step sits *after* the test steps in the same job. A failing test aborts the job before the artifact exists, and `needs: build` then skips `deploy`. There is no ordering by good intentions.
- **Test the exact bytes that ship.** The end-to-end suite runs against the assembled `publish/` directory, served at the production path, not against a dev server. The assemble script therefore also creates `preview-root/lesson-planner/` so a plain static server on port 4173 reproduces production absolute paths.
- **One switch controls the live URL.** The `ROOT_APP` environment variable is the only thing that decides which application answers `/lesson-planner/`. Cutover and rollback are both one-line edits to it.

The assemble script

```
#!/usr/bin/env bash
# scripts/assemble-site.sh – builds the exact directory tree that GitHub Pages will serve.
# Usage: scripts/assemble-site.sh legacy (or) scripts/assemble-site.sh react
set -euo pipefail
```



```

ROOT_APP="${1:-legacy}"
rm -rf publish preview-root
mkdir -p publish

if [ "$ROOT_APP" = "react" ]; then
  # Cutover state: React owns the live path; the current page stays reachable as an escape
  hatch.
  npx vite build --base /lesson-planner/ --outDir ../publish --emptyOutDir
  mkdir -p publish/legacy
  cp index.html publish/legacy/index.html
else
  # Pre-cutover state: the current page keeps the live path byte-for-byte.
  npx vite build --base /lesson-planner/next/ --outDir ../publish/next --emptyOutDir
  cp index.html publish/index.html
fi

# Reproduce the production path locally so absolute asset URLs resolve during tests.
mkdir -p preview-root
cp -R publish preview-root/lesson-planner

```

Three path details matter here, and all three are verified rather than assumed.

1. **outDir is resolved relative to the Vite project root, not the shell's working directory** — `build.outDir` is documented as "Specify the output directory (relative to project root)" [S3]. With `root: 'app', --outDir ../publish` therefore lands at `<repo>/publish`. Every other command in the script (`cp`, `mkdir`, `rm`) is cwd-relative and assumes the script is run from the repository root, which the `npm run build` script guarantees.
2. **--emptyOutDir is not required for the build to succeed, and the reason to pass it is not what most guides say.** `build.emptyOutDir` defaults to "true if `outDir` is inside `root`", and when `outDir` is *outside* `root` Vite "will emit a warning... to avoid accidentally removing important files"; setting the option explicitly "suppress[es] the warning" [S3]. So the flag both silences that warning and opts the out-of-root directory back into being emptied.
3. **Ordering matters in the react branch only.** There, the build empties `publish/` itself, so the `cp` of the legacy page into `publish/legacy/` must come after it. In the `legacy` branch the build empties only `publish/next/` while the `cp` writes `publish/index.html`, so the two are independent — the `rm -rf publish` at the top of the script already guarantees a clean tree either way.

The workflow

```

name: Build, test and deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

```

```

concurrency:
  group: pages
  cancel-in-progress: true

env:
  # The ONLY switch that moves the live URL.
  # legacy = current index.html at /lesson-planner/ , React at /lesson-planner/next/
  # react = React at /lesson-planner/ , current index.html at /lesson-planner/legacy/
  ROOT_APP: legacy

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v7 # version-sensitive
      - name: Set up Node
        uses: actions/setup-node@v7 # version-sensitive
        with:
          node-version: 22
          cache: npm
      - name: Install dependencies
        run: npm ci
      - name: Lint and format check
        run: npm run lint:ci
      - name: Type-check
        run: npm run typecheck
      - name: Unit tests
        run: npm run test:unit
      - name: Assemble publish directory
        run: bash scripts/assemble-site.sh "$ROOT_APP"
      - name: Install Playwright browsers
        run: npx playwright install --with-deps chromium
      - name: End-to-end tests
        run: npm run test:e2e
      - name: Upload Playwright report
        if: ${!cancelled()}
        uses: actions/upload-artifact@v7 # version-sensitive
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 7
      - name: Assert the three storage keys survive the build
        run: bash scripts/assert-storage-keys.sh
      - name: Configure Pages
        uses: actions/configure-pages@v6 # version-sensitive
      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5 # version-sensitive
        with:
          path: publish

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }

```

```

steps:
  - name: Deploy to GitHub Pages
    id: deployment
    uses: actions/deploy-pages@v5          # version-sensitive

```

The storage-key gate

This is nine lines that make the origin-and-keys invariant machine-checked rather than remembered. String literals survive minification, so the built bundle can be grepped.

```

#!/usr/bin/env bash
# scripts/assert-storage-keys.sh - fails the build if the React bundle stops using the live
keys.
set -euo pipefail

if [ "${ROOT_APP:-legacy}" != "react" ]; then
  echo "Pre-cutover build: staging keys expected, gate skipped."
  exit 0
fi

for key in groupLessonPlannerData groupLessonPlannerSettings paymentTemplate; do
  if ! grep -rqF "$key" publish/assets; then
    echo "FATAL: production bundle does not reference localStorage key $key"
    exit 1
  fi
done
echo "All three localStorage keys present in the production bundle."

```

Version-sensitive values

Every `uses:` tag above was verified against each action's own Releases API on **2026-08-20**. These are the values most likely to be stale when this report is read.

Action	Verified latest tag	Published	Source
actions/checkout	v7.0.1	2026-07-20	[S16]
actions/setup-node	v7.0.0	2026-07-14	[S17]
actions/configure-pages	v6.0.0	2026-03-25	[S18]
actions/upload-pages-artifact	v5.0.0	2026-04-10	[S19]
actions/deploy-pages	v5.0.0	2026-03-25	[S20]
actions/upload-artifact	v7.0.1	2026-04-10	[S23]

Two primary sources disagree, and the disagreement matters. GitHub's own *Using custom workflows with GitHub Pages* page names `configure-pages v5`, `upload-pages-artifact v4` and `deploy-pages v4` [S9]; Playwright's CI guide names `actions/checkout@v6` and `actions/setup-node@v6` [S40]. Both are one major behind the actions' own release tags. Vite's deployment page agrees with the release tags, pinning v7, v7, v6, v5 and v5 by full commit SHA [S2]. **Resolution: the action repositories' release tags**

are the source of record; the documentation prose lags. Prefer the SHA-pinning style Vite uses [S2] over floating major tags if supply-chain pinning matters; the SHAs in [S2] were current on 2026-08-20 and will themselves go stale.

`package.json` scripts these steps assume

```
{
  "scripts": {
    "dev": "vite",
    "build": "bash scripts/assemble-site.sh",
    "serve": "http-server preview-root -p 4173 -c-1",
    "typecheck": "tsc -b",
    "lint": "biome check .",
    "lint:ci": "biome ci .",
    "format": "biome check --write .",
    "test:unit": "vitest run",
    "test:e2e": "playwright test",
    "test:ui": "playwright test --ui",
    "test:update": "playwright test --update-snapshots",
    "test:trace": "playwright show-trace"
  }
}
```

None of these scripts exists in the committed tree today: **AGENTS.md** 's npm commands are aspirational, because no root `package.json` is committed and the only committed one is `docs/research/tools/package.json`, which belongs to the report pipeline [S46]. Quick win 2 is what makes them real; every reference to `npm run ...` in this report presumes it has landed.

`serve` keeps port 4173, the value `AGENTS.md:11` names and the uncommitted `playwright.config.ts` sets, and 4173 is also Vite's own `preview` default [S4], so the two conventions coincide rather than conflict. `playwright.config.ts` needs one change: `baseUrl` becomes `http://localhost:4173/lesson-planner/` — the base path is part of the URL locally too. Pre-cutover, two Playwright projects are the clean way to cover both applications: one with `baseUrl` ending `/lesson-planner/` running the existing legacy specs, one ending `/lesson-planner/next/` running the React specs. At cutover the second project's `baseUrl` loses the `next/` segment and the first is retired.

5. Development-to-cutover continuity plan

Recommendation: a staging subpath on the same origin

Publish the React build to `https://denyslystopadskyy.github.io/lesson-planner/next/` while `/lesson-planner/` continues to serve the current file unchanged. Both come from one Pages artifact, so there is no second host, no second account and no second deployment mechanism.

Option	Origin	Verdict
Staging subpath /lesson-planner/next/	Same	Recommended. Live path untouched, one artifact, realistic environment, zero extra cost
Netlify / Vercel / Cloudflare Pages preview	Different	Rejected. A different origin has a different, empty <code>localStorage</code> [S41], so it cannot be validated against real data — and if she ever opens the link thinking it is the app, she will believe her data is gone
A second GitHub repository with its own Pages site	Same host, different path	Works technically, but splits the build, the tests and the history across two repositories for no benefit over a subpath
Feature flag inside one bundle at the live path	Same	Rejected for now. Requires shipping React to the live path before it is proven — the exact risk this plan exists to avoid
Pull-request preview deployments	n/a	<code>actions/deploy-pages</code> exposes a <code>preview</code> input documented as "Pull request preview deployment (alpha only)" [S13]. Alpha, and unnecessary once a stable staging path exists

The sharp edge of choosing the same origin

Because `/lesson-planner/next/` shares the origin with the live app, **a bug in the staging build can destroy the live data**. RP-01 already documents how easily this app corrupts its own state: unguarded `JSON.parse` on load (D14), a malformed year that breaks CSV restore permanently (D4), and a stray CSV quote that silently replaces everything (D12). A half-finished React build writing to the same three keys is a realistic data-loss event, not a theoretical one.

Three rules make the same-origin choice safe. All three are design intent, not measurements.

- 1. Key isolation by default.** Read the three key names from one module that prefixes them from an environment variable — empty in the production build, `staging:` in the `/next/` build. The staging app then reads and writes `staging:groupLessonPlannerData` and its two siblings, and cannot touch the live values. The prefix constant becomes empty exactly once, at cutover, and the §4 gate script fails the build if the production bundle does not contain all three unprefixed literals.
- 2. Seed staging through the app's own front door.** Export a CSV from the live app, then import it into the staging app with Load CSV. No copy logic to write, no second code path to trust, and it exercises the CSV round-trip that RP-01 §9 flags as never proven with real data.
- 3. Export before the staging URL is ever opened on her machine, and store the file off the laptop.** Given the previous loss happened during a laptop clean, "off the laptop" means email or cloud, not the Downloads folder. If the staging app is only ever opened on the developer's machine, this rule protects nothing — but it costs one click and the failure it guards against is unrecoverable.

Phasing

Phase	What lands	Live URL serves	Publishing source
0	Commit <code>package.json</code> , <code>playwright.config.ts</code> , <code>tsconfig*.json</code> , <code>e2e/</code> ; add a test-only CI workflow	Current page, unchanged	Unchanged: branch, <code>main / (root)</code> [S46]
1	Add <code>app/</code> scaffold, <code>vite.config.ts</code> , <code>scripts/</code> , the deploy workflow with <code>R00T_APP: legacy</code> . Prove one green run. Then flip Settings → Pages → GitHub Actions	Current page, byte-identical	GitHub Actions
2	Build the React app feature by feature against <code>/lesson-planner/next/</code>	Current page, unchanged	GitHub Actions
3	Parity reached and accepted. Set <code>R00T_APP: react</code>	React app	GitHub Actions
4	After a soak period, optionally move the legacy source to <code>legacy/index.html</code>	React app	GitHub Actions

Phase 1's ordering is the crux. The flip replaces the current `main / (root)` branch publishing [S46], and GitHub's docs do not describe what happens to the live site during the switch [S8]. Because the first Actions deployment publishes the current `index.html` at the site root unchanged, the flip is **content-neutral**: if it works, nothing visible changes, and if it does not, the failure surfaces while the developer is watching rather than in the middle of a cutover. Do not flip before a green run has produced the artifact, and select **GitHub Actions**, never `main / /docs`.

One consequence of today's `main / (root)` source is worth stating because it constrains phase 1: while branch publishing is in force, **the repository root is the published site**. Adding `app/`, `scripts/`, `vite.config.ts`, `package.json` and `biome.json` to `main` therefore makes those files publicly fetchable under `/lesson-planner/...` — harmless, since they are already public on GitHub, but it also means `app/index.html` would be reachable at `/lesson-planner/app/` as an unbuilt, non-functional page. It is cosmetic, it disappears at the flip, and it is not a reason to delay committing the scaffold.

6. Cutover checklist and rollback checklist

Preconditions — all must be true before step 1

- `/lesson-planner/next/` passes the acceptance criteria for every feature in RP-01 §2 marked `core`.
- The full Playwright suite is green against `preview-root` in CI on the head of `main`.
- A CSV export taken from the live app in the last 24 hours exists **off the laptop**, and re-importing it into the staging app has been observed to restore the same groups, dates, prices and currency.
- The publishing source has already been switched from its original `main / (root)` [S46] to GitHub Actions, and at least three successful deployments have happened through it.
- The three key names in `app/src` are the unprefixed literals `groupLessonPlannerData`, `groupLessonPlannerSettings`, `paymentTemplate`, and `scripts/assert-storage-keys.sh` is wired into the workflow.

6. The teacher is not mid-task. Pick a day she is not sending payment messages.

Cutover — numbered, executable

1. Create a branch from `main`.
2. In `app/src`, set the storage-key prefix to the empty string for the production build (the one-line change described in §5 rule 1).
3. In `.github/workflows/deploy.yml`, change `R00T_APP: legacy` to `R00T_APP: react`. Change nothing else.
4. In `playwright.config.ts`, remove the `next/` segment from the React project's `baseUrl` and delete the legacy project.
5. Run locally: `npm ci`, then `npm run build`, then `npm run serve`, then `npm run test:e2e`. All green before pushing.
6. Confirm the gate by hand: `grep -rLF groupLessonPlannerData publish/assets` must print at least one file. Repeat for the other two key names.
7. Open `http://localhost:4173/lesson-planner/` in a fresh browser profile, add one group, select one date, set a price, generate the payment message, export CSV, reload, and confirm the data is still there. This is the smoke test that matters — it exercises the real keys on a throwaway profile.
8. Push the branch, open a pull request, and confirm CI is green.
9. Merge to `main`. Watch the Actions run to completion, both jobs.
10. Wait 10 minutes for the CDN, or hard-reload with cache bypass — the live response carries `cache-control: max-age=600` [S43].
11. On the developer's own machine, open `https://denyslystopadskyy.github.io/lesson-planner/`. Confirm the React app renders and, in DevTools, that `localStorage` still holds all three keys and that `groupLessonPlannerData` still parses as JSON.
12. Confirm `https://denyslystopadskyy.github.io/lesson-planner/legacy/` serves the old page. This is the escape hatch and it must be proven before the teacher is told anything.
13. Only now, tell the teacher. Give her both URLs and one sentence: if anything looks wrong, use the `/legacy/` link and message you. Ask her to take one CSV export as her first action.
14. Do not delete anything for at least two weeks.

Rollback — numbered, executable

The acceptance criterion for a successful rollback is **"all three keys are present and `groupLessonPlannerData` parses"**, not "the page loads". RP-01's D14 established that a page which loads showing nothing is exactly what data loss looks like here.

1. **Do not touch `localStorage`. Do not clear site data. Do not use Clear All Data.** Get her to stop using the app and to send you a screenshot.
2. Check the keys before changing anything. In DevTools on her machine, or by asking her to read them out, confirm whether `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate` are still present. Record the answer; it determines whether this is a display problem or a data problem.

3. Tell her to use `https://denyslystopadskyy.github.io/lesson-planner/legacy/` immediately. It reads the same three keys on the same origin, so if the data is intact she can keep working within a minute. This step comes before any repository work.
4. **Fast path — bad deployment, good commit.** GitHub → Actions → the last known-good run → *Re-run all jobs*. Re-runs reuse the original commit SHA, are available for **30 days** after the original run, and are capped at **50 re-runs** per run [S11]. Note that `upload-pages-artifact` sets `retention-days: 1` by default [S14], so the original artifact has almost certainly expired — the re-run rebuilds from the same SHA rather than redeploying stored bytes.
5. **Durable path — bad commit.** On `main`, change `R00T_APP: react` back to `R00T_APP: legacy`, commit, push. This is a one-line edit and it restores the current page to `/lesson-planner/` while leaving the React build available at `/lesson-planner/next/` for diagnosis. Prefer this over `git revert` of the whole merge, which would also undo the React work.
6. Wait for both jobs to go green, then wait out the 10-minute cache [S43], then verify `https://denyslystopadskyy.github.io/lesson-planner/` serves the old page.
7. Verify the invariant again: all three keys present, `groupLessonPlannerData` parses. If yes, the incident is closed for her.
8. **If a key is missing or unparseable**, stop and restore from the CSV export taken in the preconditions, using Load CSV in the legacy app. Note two known gaps from RP-01 before promising a full restore: the export omits `paymentTemplate` (D16), so a customised template is not in the CSV; and a single malformed date makes the whole import throw (D4).
9. Write down what happened before starting the fix. The next cutover attempt needs the failure mode, not the memory of it.

What must never be part of a rollback

Changing the repository name, the account name, or adding or removing a custom domain. Each changes the origin and makes her data unreachable [S41] — a rollback that does any of these converts a bad deployment into permanent data loss.

7. Minimal tooling configuration, plus an explicit "not worth it here" list

TypeScript

Recommend `typescript ~6.0.x`, and mirror `create-vite`'s project-reference layout [S28][S29] rather than inventing one. TypeScript 6.0 (released 2026-03-23) makes this cheaper than it used to be: it states outright that "strict is now true by default", that "the new default `module` is `esnext`", and that the default `target` is now a floating latest-spec value, currently `es2025` [S22].

The existing `tsconfig.json` in the main checkout will break on TypeScript 6. It sets `"module": "commonjs"` and `"moduleResolution": "node"`, and TypeScript 6.0 removed `--moduleResolution node` along with `--module amd/umd/systemjs`, `--baseUrl` and `--outFile` [S22]. It also only covers `e2e/**/*.ts` and `playwright.config.ts`, so it does not type-check application source at all, and its CommonJS module settings are wrong for a Vite ESM app. Do not extend it — add siblings.

File	Purpose	Key settings
tsconfig.json	Solution file only	files: [], references to the three below
tsconfig.app.json	React source	include: ["app/src"], jsx: "react-jsx", module: "esnext", moduleResolution: "bundler", types: ["vite/client"], noEmit: true, verbatimModuleSyntax: true
tsconfig.node.json	Build config	include: ["vite.config.ts"], module: "nodenext", types: ["node"], noEmit: true
tsconfig.e2e.json	Existing Playwright suite	The current file, with module changed to nodenext and moduleResolution: "node" deleted

Set "strict": true explicitly in each even though TS 6.0 defaults it on [S22] — one word that makes the intent survive a version downgrade, and it is what AGENTS.md:18 asks for.

Linting and formatting

Recommend **Biome 2.5.9, pinned exactly**. Biome's own guide recommends `npm i -D -E @biomejs/biome` and its versioning page explains why: "Fixes to lint rules, formatting layouts, etc. might prevent your scripts from passing", so exact pinning is deliberate, not paranoia [S25]. One binary, one `biome.json`, and TypeScript, TSX, JSX, JSON, JSONC and CSS are all supported for parsing, formatting and linting [S26].

```
{
  "$schema": "https://biomejs.dev/schemas/2.5.9/schema.json",
  "files": { "includes": ["app/**", "e2e/**", "*.ts", "*.json"] },
  "formatter": { "indentStyle": "space", "indentWidth": 2 },
  "linter": {
    "rules": {
      "recommended": true,
      "style": {
        "useFilenamingConvention": {
          "level": "error",
          "options": { "filenameCases": ["kebab-case"] }
        }
      }
    }
  }
}
```

`formatter.indentStyle` defaults to "tab" [S25], so the "space" line is required to satisfy AGENTS.md:20. `style/useFilenamingConvention` is **not** enabled by default and must be turned on explicitly [S44]; setting `filenameCases: ["kebab-case"]` makes AGENTS.md's kebab-case filename rule machine-enforced instead of aspirational. The `$schema` version string must be bumped alongside the pinned package version.

Alternative	Dev deps	Config files	Why not chosen
ESLint 10.8.1 + typescript-eslint 8.67.0 + Prettier 3.9.6	4 or more, plus React hooks plugin	<code>eslint.config.js</code> + <code>.prettierrc</code>	Largest maintenance surface. ESLint v10 removed legacy <code>.eslintrc</code> entirely — "Starting with ESLint v10.0.0, the old configuration format is no longer supported" [S45] — so any older guide found online is now wrong. typescript-eslint peers <code>typescript >=4.8.4 <6.1.0</code> [S24], which caps TypeScript at the 6.0 line and rules out 7.x. Justified on a large team codebase; not on a 1,473-line single-screen app
oxlint 1.79.0 + oxfmt 0.64.0	2	<code>.oxlintrc.json</code> + <code>oxfmt config</code>	oxlint is the default linter in <code>create-vite</code> 's own <code>react-ts</code> template [S28] and is genuinely good, with type-aware rules that "leverage the native Go port of the TypeScript compiler" [S27] — the one path that does not pin TypeScript backwards. But the formatter half, <code>oxfmt</code> , is at 0.64.0 , pre-1.0 [S36], so formatting output can shift between releases
Biome 2.5.9	1	<code>biome.json</code>	Chosen. One tool, stable major, does both jobs. Caveat, stated honestly: Biome documents JavaScript ES2024 and "TypeScript version 5.9" language support [S26], so TypeScript 6.0-or-later-only syntax may not parse. Nothing in this app needs it

If type-aware lint rules become desirable later, add oxlint alongside Biome's formatter rather than migrating to typescript-eslint — it is the option that does not force a TypeScript downgrade.

Testing

Vitest 4.1.11 for **pure functions only**: money arithmetic, `YYYY-MM` and `YYYY-MM-DD` key helpers, CSV serialise and parse, and the storage-load guard. These are exactly where RP-01 located D4, D12 and D20, and they are the parts of the app that have never had a test. Playwright keeps everything that touches the DOM. RP-01 §5 notes that extracting `services` and `utils` into pure modules is "the cheapest early win in a React migration" — Vitest is what makes that extraction pay.

Not worth it here — and why

Each of these is standard advice that is wrong at this scale. The scale is one screen, roughly 1,500 lines, one user, one maintainer working in personal time, and zero runtime dependencies today.

Rejected	Reason
Husky, lint-staged, commitlint	Pre-commit machinery for a solo developer who is also the only reviewer. CI already runs <code>biome ci</code> and <code>tsc -b</code> . Adds an install step that breaks on every fresh clone
Storybook	Component catalogue for a design system that does not exist and a team of one
React Testing Library plus jsdom component tests	Duplicates the Playwright suite's assertions in a fake DOM. Two suites asserting the same behaviour is worse than one, because they disagree

Rejected	Reason
Coverage thresholds, Codecov	A gate that gets lowered rather than met. Meaningful coverage here is "the money maths and the CSV parser have tests", which is checkable by reading the test directory
Bundle-size budgets, size-limit, bundle analyzer in CI	Two dependencies and a chunk graph. <code>npm vite build</code> already prints chunk sizes on every run
semantic-release, changesets, conventional commits	Versioned releases for an unversioned private app with one consumer who never sees a version number
Renovate or Dependabot version-bump PRs	6 to 8 dev dependencies, none of which ship to the browser. A quarterly <code>npm outdated</code> costs less than the PR stream. GitHub's free security alerts are a repository setting, cost nothing and produce no noise — take those
react-router or any router	See §3. Zero URL usage today and no stable entity ids to route to
Redux, Zustand, TanStack Query	No server, no async data, one screen. <code>useState</code> plus one reducer for modal state is the whole requirement
Tailwind, CSS-in-JS, a component library	RP-01 §5 measured 224 lines of CSS with one custom property. Port it as-is and delete the six dead rule groups on the way
<code>@vitejs/plugin-legacy</code>	See §8. Vite's default target already reaches browsers from 2023
A Playwright browser matrix across Chromium, Firefox and WebKit	Triples CI time for one user on one unknown browser. Run Chromium until her browser is known, then match it
A Node version matrix in CI	One runtime, one deployment. Pin one Node major
Production source maps	Default is <code>false</code> [S3]. Leave it. They would roughly double deployed bytes for a debugging convenience that <code>vite preview</code> already provides locally
A monorepo tool	One package

Deliberately **not** rejected, just out of scope for this report: a service worker for offline use. For a teacher who works from a laptop that has been wiped once, offline capability and a durable local cache have real value, but a service worker also introduces a stale-build failure mode that can strand her on old code — which is precisely the class of problem §6 exists to avoid. It needs its own decision, not a footnote here.

8. Browser support and build targets

Recommendation: change nothing. Ship Vite 8's default target and add no legacy plugin.

Vite 8's `build.target` defaults to `'baseline-widely-available'`, documented as "the minimum browser versions compatible with Baseline Widely Available as of a date fixed for each major release (2026-01-01 for this major)", resolving to `['chrome111', 'edge111', 'firefox114', 'safari16.4', 'ios16.4']` [S3]. Chrome 111 and Safari 16.4 both shipped in early 2023, so the default already covers roughly three and a half years of consumer hardware without any configuration.

Concern	Position
Desktop laptop, unknown vendor	The default target covers any browser kept even loosely up to date. No action
Phone	<code>ios16.4</code> is in the default target [S3]. No action for the build; the layout is a separate problem — see below
APIs the app already requires	<code>localStorage</code> , <code>Intl.NumberFormat</code> , <code>navigator.clipboard</code> , <code>FileReader</code> , <code>Blob</code> , <code>URL.createObjectURL</code> (RP-01 §5). All predate the default target by years, so the migration does not raise the browser floor
ES modules	Vite's built <code>index.html</code> loads a <code>type="module"</code> script with no <code>nomodule</code> fallback. Every browser in the default target supports ESM, so this is only a risk on a browser far older than the target
Minifier	<code>build.minify</code> defaults to <code>'oxc'</code> for the client build [S3]. Leave it
Transpile-down further	Only if her device turns out to be below the baseline. Set <code>build.target</code> explicitly then — do not reach for <code>@vitejs/plugin-legacy</code> , which exists for the far older browsers this app will never need to serve

The unknown that actually matters

The end user's browser and operating system are **TBD**. This is not a number that can be inferred from "non-technical user on a personal laptop" — it is a fact about a specific device. It is resolved by asking her, or by reading the `User-Agent` string from any support exchange, or by having her open the app and read out what the About dialog says. Until it is known, one specific risk stands: if she is on an operating system that cannot reach Safari 16.4 or an equivalent, the default target ships syntax her browser cannot parse, and the symptom is a blank page — the same symptom as a `base` misconfiguration, which will make it easy to misdiagnose.

Two facts about phone use that are more important than the build target

First, `localStorage` is **per-origin and per-browser** [S41]. Using the app on a laptop and on a phone gives **two separate, silently divergent datasets**. There is no sync and nothing in the app hints at this. If phone use is genuinely wanted rather than incidental, that is a product requirement this migration does not address, and CSV export-and-import is the only bridge that exists.

Second, the phone layout is already partly broken and a build target cannot fix it. RP-01 §6 measured that at 375 px `.calendar-controls` computes `flex-wrap: nowrap`, pushing `#clearMonthBtn` to `x=394` in a 375 px viewport and clipping the ◀ button at the left edge, with both unreachable without scrolling inside the modal. Anyone treating "possible phone use" as supported today should read that row first.

9. Risks and unknowns

#	Risk or unknown	Impact	What resolves it
R1	A staging build on the same origin can destroy live data. Chosen deliberately (§5) because every alternative loses access to her real data	Critical	The three rules in §5: key prefixing, seeding via CSV, and an export stored off the laptop before the staging URL is opened on her machine
R2	Every GitHub Pages site under <code>denyslstopadskyy.github.io</code> shares one origin and therefore one <code>localStorage</code> namespace [S41]. Any other project published from that account can read the teacher's data, and a key-name collision would corrupt it	High	Do not publish anything else from that account that writes <code>localStorage</code> , or accept that the app's data is not isolated. Moving the app to another repository does not help — same host
R3	The whole plan is gated on one manual repository-owner action — switching Settings → Pages to GitHub Actions. The current source is branch-based, <code>main</code> / (root) [S46], and nothing in CI can change it	High, because it is a single point of failure outside the code	Confirm the owner will make the change before phase 1 starts. If they will not, the only branch-based fallback is committing built output to the root of <code>main</code> , contrary to <code>AGENTS.md:36</code> — a decision, not a workaround
R4	Flipping the publishing source is not documented as to what happens to the live site during the switch [S8]	Medium	Sequence it as §5 phase 1 does: prove a green run first, and make the first Actions deployment publish byte-identical content so the flip is content-neutral
R4a	Choosing <code>main</code> / <code>/docs</code> instead of GitHub Actions at the Settings screen would publish the research reports in place of the application [S8][S46]	High if it happens, trivially avoidable	Name the correct option in the runbook: GitHub Actions. Never the <code>/docs</code> folder for this repository
R5	Bundle wire weight after migration is TBD . RP-01 measured 13,610 bytes gzipped in one request today; React will be larger in at least three requests	Medium	Run <code>npx vite build</code> once and read the reported chunk sizes. Cannot be measured before the app exists
R6	The end user's browser and OS are TBD (§8)	Medium — a below-baseline browser fails identically to a <code>base</code> misconfiguration	Ask her, or read a <code>User-Agent</code> from any support exchange

#	Risk or unknown	Impact	What resolves it
R7	Action major versions go stale fast, and the official docs already disagree with the release tags — GitHub's own Pages workflow page names v5/v4/v4 while the actions ship v6/v5/v5 [S9] versus [S18][S19][S20]	Medium	Re-check each action's Releases API before pasting the §4 YAML. Every <code>uses:</code> line is marked version-sensitive
R8	<code>typescript-eslint</code> peers <code>typescript</code> <6.1.0 [S24] and TypeScript 7.0 ships no stable API [S21], so the linting choice and the TypeScript version are coupled	Medium	Recommending Biome decouples them, because Biome does not consume the TypeScript API. Revisit when TypeScript 7.1 ships the new API [S21]
R9	Biome documents "TypeScript version 5.9" language support [S26] while TypeScript 6.0 and 7.0 exist	Low here, rising over time	Re-read Biome's language-support page at each Biome upgrade. Nothing in this app uses TS 6-or-later-only syntax
R10	<code>oxfmt</code> is at 0.64.0, pre-1.0 [S36]	Low — only matters if <code>oxlint</code> plus <code>oxfmt</code> is chosen over Biome	Wait for a 1.x release before adopting <code>oxfmt</code> as the sole formatter
R11	Whether Jekyll processing applies to an Actions-deployed Pages artifact is TBD . Moot today because Vite emits <code>assets/</code> , not an underscore-prefixed directory [S3]	Low	Publish one file whose name starts with an underscore and check whether it is served
R12	In the <code>react</code> branch, <code>--emptyOutDir</code> empties <code>publish/</code> itself, so reordering the build and the legacy <code>cp</code> would silently drop the escape-hatch page [S3]	Low but silent	The script builds before copying. <code>scripts/assert-storage-keys.sh</code> and the Playwright run against <code>preview-root</code> would both catch an empty or partial tree before deployment
R13	The assemble script mixes a root-relative Vite <code>outDir</code> with cwd-relative shell commands. Verified consistent — <code>build.outDir</code> is documented as relative to the project root [S3] — but it will break if anyone changes <code>root</code> , moves the script, or invokes it from a subdirectory	Low, and would fail loudly	Run <code>npm run build</code> once from the repository root and confirm <code>publish/index.html</code> exists; add a <code>cd</code> guard to the script if it is ever called from elsewhere
R14	Pages soft limit of 10 builds per hour [S10]	Low — reachable during a debugging session	Batch pushes, or debug locally with <code>npm run build</code> and <code>npm run serve</code>

#	Risk or unknown	Impact	What resolves it
R15	A service worker for offline use is out of scope but genuinely valuable for this user, and adding one later introduces a stale-build failure mode	Deferred	A separate decision with its own report. Do not add one as part of this migration
R16	The personal data exposure RP-01 recorded at <code>index.html:387-392</code> and <code>index.html:400</code> follows the code into the React build unless it is removed first	Critical, and inherited	RP-01's first quick win. Deal with it before the template string is copied into a <code>.ts</code> file, where it will also be in the built bundle

All TBD items in one place

TBD	Reason	Resolver
GitHub Pages publishing source — RESOLVED , not a TBD	Was: Settings not publicly readable	Resolved 2026-08-20 by public-evidence inference: branch publishing, <code>main / (root)</code> [S46]
Failure mode when a Pages workflow runs while the source is still "Deploy from a branch"	Not documented [S8] [S9]	One trial run
Whether Jekyll runs for an Actions-deployed artifact	Not stated on the pages fetched [S9]	Publish an underscore-prefixed file and check
Built bundle size, gzipped, and request count	Nothing has been built	<code>npx vite build</code> , read the chunk report
The end user's browser and operating system	Not knowable from the repository	Ask her, or read a <code>User-Agent</code> string
Whether Vite normalises a <code>base</code> value given without a trailing slash	Not stated on the fetched config page [S1]	Build once with <code>base: '/lesson-planner'</code> and inspect the emitted asset URLs in <code>publish/index.html</code>

10. Sources

#	Title	URL	Accessed	Supports
S1	Vite — Shared Options (base)	https://vite.dev/config/shared-options.html	2026-08-20	<code>base</code> type <code>string</code> , default <code>/</code> , documented forms <code>/foo/</code> , a full URL, or empty <code>./</code> for embedded deployment
S2	Vite — Deploying a Static Site	https://vite.dev/guide/static-deploy.html	2026-08-20	Set <code>base</code> to <code><REP0>/</code> for a project site; the complete Pages workflow YAML with SHA-pinned <code>v7/v7/v6/v5/v5</code> actions; <code>dist</code> is the upload path
S3	Vite — Build Options	https://vite.dev/config/build-options.html	2026-08-20	<code>build.target</code> default <code>baseline-widely-available</code> resolving to <code>chrome111/edge111/firefox114/safari16.4/ios16.4</code> , fixed at 2026-01-01 for v8; <code>outDir</code> default <code>dist</code> and "relative to project root"; <code>emptyOutDir</code> default "true if outDir is inside root", warning plus opt-in when outside; <code>sourcemap</code> <code>false</code> ; <code>minify</code> <code>oxc</code> for the client build
S4	Vite — Preview Options	https://vite.dev/config/preview-options.html	2026-08-20	<code>preview.port</code> default 4173, the same port <code>AGENTS.md:11</code> names and the uncommitted <code>playwright.config.ts</code> sets
S5	Vite — Command Line Interface	https://vite.dev/guide/cli.html	2026-08-20	<code>vite build</code> flags <code>--base</code> , <code>--outDir</code> , <code>--emptyOutDir</code> ("Force empty outDir when it's outside of root"), <code>--target</code> , <code>--minify</code>

#	Title	URL	Accessed	Supports
S6	Vite — Getting Started	https://vite.dev/guide/	2026-08-20	Node.js 20.19+ or 22.12+ requirement; <code>npm create vite@latest</code> ; the <code>react-ts</code> template exists
S7	React Blog — Sunsetting Create React App	https://react.dev/blog/2025/02/14/sunsetting-create-react-app	2026-08-20	Dated 2025-02-14: "Today, we're deprecating Create React App for new apps"; recommends Next.js, React Router, Expo, then Vite, Parcel, RSBUILD
S8	GitHub Docs — Configuring a publishing source for your GitHub Pages site	https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site	2026-08-20	The two publishing sources and the exact Settings → Pages steps; branch publishing allows root or <code>/docs</code> ; no statement on what happens when switching

#	Title	URL	Accessed	Supports
S9	GitHub Docs — Using custom workflows with GitHub Pages	https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages	2026-08-20	Required permissions contents: read, pages: write, id-token: write; environment github-pages; the needs requirement; the stale v5/v4/v4 action versions in prose
S10	GitHub Docs — GitHub Pages limits	https://docs.github.com/en/pages/getting-started-with-github-pages/github-pages-limits	2026-08-20	1 GB published site, 100 GB/month soft bandwidth, 10 builds/hour soft limit, 10-minute deployment timeout

#	Title	URL	Accessed	Supports
S11	GitHub Docs — Re-running workflows and jobs	https://docs.github.com/en/actions/how-to/manage-workflow-runs/re-run-workflows-and-jobs	2026-08-20	Re-runs use the original GITHUB_SHA , are available for 30 days, and are capped at 50 per run — the rollback fast path
S12	GitHub Docs — Removing workflow artifacts	https://docs.github.com/en/actions/how-to/manage-workflow-runs/remove-workflow-artifacts	2026-08-20	Default 90-day artifact retention, against which upload-pages-artifact 's 1-day override is notable

#	Title	URL	Accessed	Supports
S13	actions/deploy-pages — README	https://github.com/actions/deploy-pages	2026-08-20	Required <code>pages: write</code> and <code>id-token: write</code> ; <code>id-token "necessary to request the OIDC JWT token"</code> ; <code>environment: github-pages</code> ; <code>artifact_name default github-pages</code> ; <code>timeout default 600000 ms</code> ; <code>alpha preview input</code>
S14	actions/upload-pages-artifact — action.yml	https://raw.githubusercontent.com/actions/upload-pages-artifact/main/action.yml	2026-08-20	Inputs <code>name default github-pages</code> , <code>path default _site/</code> and <code>required, retention-days default 1</code> , <code>include-hidden-files default false</code>
S15	actions/configure-pages — action.yml	https://raw.githubusercontent.com/actions/configure-pages/main/action.yml	2026-08-20	Outputs <code>base_url</code> , <code>origin</code> , <code>host</code> and <code>base_path</code> ; <code>base_path</code> documented as <code>/my-repo</code> with no trailing slash

#	Title	URL	Accessed	Supports
S16	GitHub Releases API — actions/checkout latest	https://api.github.com/repos/actions/checkout/releases/latest	2026-08-20	v7.0.1, published 2026-07-20
S17	GitHub Releases API — actions/setup-node latest	https://api.github.com/repos/actions/setup-node/releases/latest	2026-08-20	v7.0.0, published 2026-07-14
S18	GitHub Releases API — actions/configure-pages latest	https://api.github.com/repos/actions/configure-pages/releases/latest	2026-08-20	v6.0.0, published 2026-03-25

#	Title	URL	Accessed	Supports
S19	GitHub Releases API — actions/upload-pages-artifact latest	https://api.github.com/repos/actions/upload-pages-artifact/releases/latest	2026-08-20	v5.0.0, published 2026-04-10
S20	GitHub Releases API — actions/deploy-pages latest	https://api.github.com/repos/actions/deploy-pages/releases/latest	2026-08-20	v5.0.0, published 2026-03-25
S21	TypeScript Blog — Announcing TypeScript 7.0	https://devblogs.microsoft.com/typescript/announcing-typescript-7-0/	2026-08-20	Released 2026-07-08, native Go port, 8x-12x faster; "TypeScript 7.0 is no longer ship[ping] with an API"; 7.1 expected to ship a new API; deprecated options removed

#	Title	URL	Accessed	Supports
S22	TypeScript Blog — Announcing TypeScript 6.0	https://devblogs.microsoft.com/typescript/announcing-typescript-6-0/	2026-08-20	Released 2026-03-23; "strict is now true by default"; module defaults to esnext; target floats to es2025; --moduleResolution node, --outFile, --baseUrl removed
S23	GitHub Releases API — actions/upload-artifact latest	https://api.github.com/repos/actions/upload-artifact/releases/latest	2026-08-20	v7.0.1, published 2026-04-10, against Playwright's documented v4
S24	typescript-eslint — Dependency Versions	https://typescript-eslint.io/users/dependency-versions/	2026-08-20	Supported TypeScript range <code>>=4.8.4 <6.1.0</code> ; TypeScript 7 not supported; mirrors DefinitelyTyped's two-year window

#	Title	URL	Accessed	Supports
S25	Biome — Getting Started	https://biomejs.dev/guides/getting-started/	2026-08-20	Biome formats and lints; install <code>npm i -D -E @biomejs/biome</code> ; config <code>biome.json</code> ; commands <code>format</code> , <code>lint</code> , <code>check</code> , <code>ci</code> ; <code>formatter.indentStyle</code> default <code>tab</code> , <code>indentWidth</code> default <code>2</code>
S26	Biome — Language Support	https://biomejs.dev/internal/s/language-support/	2026-08-20	TypeScript, TSX, JSX, JSON, JSONC and CSS supported for parsing, formatting and linting; stated TypeScript language support level 5.9; HTML formatting experimental
S27	Oxc — Linter (oxlint)	https://oxc.rs/docs/guide/usage/linter.html	2026-08-20	oxlint lints only; supports TS and TSX; type-aware rules "leverage the native Go port of the TypeScript compiler"; oxfmt is the separate formatter

#	Title	URL	Accessed	Supports
S28	create-vite — template-react-ts/package.json	https://raw.githubusercontent.com/vitejs/vite/main/packages/create-vite/template-react-ts/package.json	2026-08-20	Official template pins <code>react/react-dom ^19.2.8</code> , <code>@vitejs/plugin-react ^6.0.5</code> , <code>oxlint ^1.78.0</code> , <code>typescript ~6.0.2</code> , <code>vite ^8.2.1</code> ; build script <code>tsc -b && vite build</code>
S29	create-vite — template-react-ts/tsconfig.app.json	https://raw.githubusercontent.com/vitejs/vite/main/packages/create-vite/template-react-ts/tsconfig.app.json	2026-08-20	The reference app tsconfig: <code>jsx: react-jsx</code> , <code>module: esnext</code> , <code>moduleResolution: bundler</code> , <code>types: ["vite/client"]</code> , <code>noEmit</code> , <code>verbatimModuleSyntax</code> , <code>include: ["src"]</code>

#	Title	URL	Accessed	Supports
S30	npm registry — vite	https://registry.npmjs.org/vite	2026-08-20	latest 8.2.2, previous 7.3.6
S31	npm registry — react	https://registry.npmjs.org/react/latest	2026-08-20	19.2.8
S32	npm registry — typescript	https://registry.npmjs.org/typescript	2026-08-20	latest 7.0.2, 6.0.x line current alongside it
S33	npm registry — vitest	https://registry.npmjs.org/vitest	2026-08-20	4.1.11, peers vite ^6.0.0 ^7.0.0 ^8.0.0
S34	npm registry — @biomejs/biome	https://registry.npmjs.org/@biomejs/biome	2026-08-20	2.5.9
S35	npm registry — oxlint	https://registry.npmjs.org/oxlint	2026-08-20	1.79.0

#	Title	URL	Accessed	Supports
S36	npm registry — oxfmt	https://registry.npmjs.org/oxfmt	2026-08-20	0.64.0, pre-1.0
S37	npm registry — @playwright/test	https://registry.npmjs.org/@playwright/test/latest	2026-08-20	1.62.1, against the <code>^1.51.0</code> in the uncommitted <code>package.json</code>
S38	npm registry — eslint	https://registry.npmjs.org/eslint/latest	2026-08-20	10.8.1, engines Node <code>^20.19.0 ^22.13.0 >=24</code>
S39	npm registry — @vitejs/plugin-react	https://registry.npmjs.org/@vitejs/plugin-react/latest	2026-08-20	6.1.0, peers <code>vite ^8.0.0</code>
S40	Playwright — Setting up CI	https://playwright.dev/docs/ci-intro	2026-08-20	The reference GitHub Actions workflow, <code>npx playwright install --with-deps</code> , and the stale <code>actions/checkout@v6</code> and <code>actions/setup-node@v6</code> pins

#	Title	URL	Accessed	Supports
S41	MDN — Window.localStorage	https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage	2026-08-20	<code>localStorage</code> is scoped per origin, which is the whole basis of the origin-and-keys invariant and of the shared-namespace risk R2
S42	GitHub Docs — Creating a custom 404 page for your GitHub Pages site	https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-custom-404-page-for-your-github-pages-site	2026-08-20	A <code>404.html</code> in the publishing source is served for missing paths — the SPA-fallback mechanism discussed and declined in §3

#	Title	URL	Accessed	Supports
S43	Live application HTTP responses, measured	https://denyslystopadskyy.github.io/lesson-planner/	2026-08-20	HTTP 200, <code>last-modified: Mon, 13 Oct 2025 14:16:26 GMT</code> , <code>etag: "68ed09ba-e519"</code> , <code>cache-control: max-age=600</code> ; <code>/assets/index.js</code> returns 404 and <code>denyslystopadskyy.github.io/</code> returns 404, confirming project-scoped subpath hosting
S44	Biome — <code>useFilenamingConvention</code>	https://biomejs.dev/linter/rules/use-filenaming-convention/	2026-08-20	Rule id <code>style/useFilenamingConvention</code> , not recommended by default, <code>filenameCases</code> accepts <code>kebab-case</code>
S45	ESLint — Migrate to v10.0.0	https://eslint.org/docs/latest/use/migrate-to-10.0.0	2026-08-20	"Starting with ESLint v10.0.0, the old configuration format is no longer supported"; Node 20.19+/22.13+/24+
S46	Repository branch, tree and deploy-timing inspection — orchestrator-verified addendum	https://github.com/denyslystopadskyy/lesson-planner	2026-08-20	Publishing source is "Deploy from a branch", <code>main / (root)</code> : no <code>gh-pages</code> branch on the remote, live <code>last-modified</code> 42 seconds after commit <code>cf69aa4</code> on <code>main</code> , <code>main</code> tree is only <code>index.html</code> , <code>AGENTS.md</code> , <code>LICENSE</code> , <code>.gitignore</code> , no <code>.github/</code> anywhere, served bytes identical to <code>main:index.html</code> . Also: no root <code>package.json</code> is committed, so every <code>AGENTS.md</code> npm script is aspirational

11. Quick wins

Four items qualify. Each is shippable today, none requires the publishing source to change, a bundler to exist, or the React work to have started, and none is blocked by an unresolved TBD or an open decision in this report. Deliberately excluded: scaffolding Vite (technically unblocked, but it is the migration itself, not a quick win), switching the publishing source to GitHub Actions (a manual owner action with live-site consequences, sequenced in §5 phase 1, not a quick win), and anything that presumes the linter choice in §7 has been accepted.

Ranks 1 and 2 must land in that order — the workflow needs the `package.json` the first item commits. Everything after is independent.

Rank	Quick win	Effort	Impact	Basis of ranking
1	Commit the uncommitted <code>package.json</code> , <code>playwright.config.ts</code> , <code>tsconfig.json</code> and <code>e2e/</code>	S	High	AGENTS.md:9–15 documents <code>npm install</code> , <code>npm run serve</code> and <code>npm run test:e2e</code> ; none exists in the committed tree, and the only committed <code>package.json</code> belongs to the report pipeline [S46]. Nothing else in this report can be automated until this lands, which is what puts it first despite being invisible to the end user
2	Add a CI workflow that runs the Playwright suite on push and pull request, with no deploy	S	High	Today any push to <code>main</code> reaches the live page the teacher depends on with zero automated check, because branch publishing serves <code>main</code> root directly [S46]. One file, no repository setting changed, no risk to the live site, and it is the safety net every later phase assumes exists
3	Add <code>dist</code> , <code>publish</code> , <code>preview-root</code> and the TypeScript build-info cache to <code>.gitignore</code>	XS	Medium	AGENTS.md:36 forbids committing generated artifacts, and the current <code>.gitignore</code> lists none of the four directories a bundled build will create. Two minutes now prevents a 300-file accidental commit later
4	Reference a favicon with a subpath-relative href	XS	Medium	RP-01 D23: a <code>favicon.ico</code> 404 logs a console error on every single load, requested from the domain root instead of the project path. It is the same absolute-versus-subpath bug class as §2's <code>base</code> failure, it is user-visible in the browser tab, and it is one line plus one small file

PROMPT QW-1: Commit the toolchain files that AGENTS.md already documents
Context: Repo lesson-planner. On the main branch the committed application tree is only `index.html`, `AGENTS.md`, `LICENSE` and `.gitignore`, plus a `docs/research` tree added by the research programme whose `docs/research/tools/package.json` belongs to the report build pipeline and is unrelated to the app. There is NO root `package.json`, so AGENTS.md lines 9 to 15 document `npm install`, `npm run serve` on port 4173, `npm run test:e2e`, `npm run test:ui`, `npm run test:update` and `npm run test:trace`, and AGENTS.md line 5 documents an `e2e/` directory, none of which is runnable today. They DO exist as uncommitted files in the developer's main checkout: `package.json` with `devDependencies` `@faker-js/faker`, `@playwright/test`, `http-server` and `typescript` plus those exact scripts, `playwright.config.ts` with `testDir e2e/features`,

baseUrl http://localhost:4173, timezoneId UTC and a webServer block, tsconfig.json covering e2e/**/*ts, and an e2e/ suite. RP-01 recorded this mismatch as a prior-art defect.

Task: Commit package.json, package-lock.json, playwright.config.ts, tsconfig.json and the e2e/ directory so that the commands AGENTS.md documents actually work from a fresh clone. Generate package-lock.json by running npm install once. Do not change any script name, the port 4173, the baseUrl or the testDir, because AGENTS.md line 38 requires npm run serve and playwright.config.ts to stay aligned.

Constraints: Do not modify index.html. Do not modify AGENTS.md. Do not modify the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape; this prompt adds tooling files only and must not touch application code. Do not commit node_modules, test-results or playwright-report, all of which .gitignore already covers. Do not add React, Vite, a bundler, a linter or any dependency that is not already present in the uncommitted package.json. Note that the uncommitted index.html in the main checkout differs from the committed one; do NOT bring that file across in this change.

Acceptance criteria: A fresh clone followed by npm ci then npm run test:e2e runs the suite without a missing-file or missing-script error. git status is clean after the commit apart from ignored paths. The committed index.html is byte-identical to the version at commit 211eb6b, which is 58649 bytes and 1473 lines. npm run serve starts a server on port 4173.

Verification: Run wc -c index.html and confirm 58649. Run npm ci then npm run test:e2e and record the pass and fail counts. Run npm run serve and confirm http://localhost:4173 responds. Run git status --porcelain and confirm no unexpected untracked file.

PROMPT QW-2: Add a test-only CI workflow that gates pull requests

Context: Repo lesson-planner. There is no .github directory and no workflow of any kind. GitHub Pages is configured as Deploy from a branch with source main and the repository root, verified 2026-08-20, so every push to main publishes the repository root directly to https://denyslystopadskyy.github.io/lesson-planner/ with no automated check of any kind. The end user depends on that page for real work and her data lives only in her browser localStorage. A Playwright suite exists in the main checkout under e2e/ with playwright.config.ts pinning baseUrl http://localhost:4173 and a webServer command of npm run serve. This prompt creates a new file and depends on QW-1 having landed the package.json that defines npm run serve and npm run test:e2e.

Task: Create a new file .github/workflows/test.yml defining a single job named test that runs on ubuntu-latest, triggered on push to main and on pull_request. Steps: checkout, set up Node 22 with npm cache enabled, npm ci, npx playwright install --with-deps chromium, npm run test:e2e, and an always-run upload of the playwright-report directory as an artifact with a 7 day retention. Set a top-level permissions block of contents: read only. Do NOT add any GitHub Pages step, any deploy step, any pages or id-token permission, or any environment block.

Constraints: This workflow must not deploy anything and must not change the GitHub Pages publishing source or any repository setting. Do not modify index.html, AGENTS.md or .gitignore. Do not commit any generated artifact. The action major versions are version-sensitive: verify actions/checkout, actions/setup-node and actions/upload-artifact against each action's Releases API before pinning, rather than copying versions from any tutorial. As of 2026-08-20 the current tags are actions/checkout v7.0.1, actions/setup-node v7.0.0 and actions/upload-artifact v7.0.1.

Acceptance criteria: The file .github/workflows/test.yml exists and is valid YAML. Opening a pull request produces exactly one workflow run with one job named test. The run fails if any Playwright spec fails and succeeds if all pass. No GitHub Pages deployment is created by this workflow, and https://denyslystopadskyy.github.io/lesson-planner/ still returns HTTP 200 serving the same content as before, with an unchanged etag.

Verification: Push the branch and open a pull request. Confirm the Actions tab shows the test job and that it goes green. Then deliberately break one assertion in one spec, push, and confirm the run goes red and the playwright-report artifact is still uploaded. Revert the

break. Finally run `curl -sI https://denyslystopadskyy.github.io/lesson-planner/` and confirm HTTP 200 with the same etag value as before the change.

PROMPT QW-3: Ignore the build output directories before any build tooling arrives
Context: Repo lesson-planner. The committed `.gitignore` contains exactly five entries under one comment: `node_modules`, `playwright-report`, `test-results`, `e2e/.cache` and `.playwright-cache`. `AGENTS.md` line 36 states that generated output must never be committed. No bundler exists yet, so none of the directories a bundled build produces are ignored. A Vite build defaults its output directory to `dist`, TypeScript project references write a build-info cache under `node_modules/.tmp`, and the deployment plan in this repository's RP-04 report introduces `publish` and `preview-root` as assembled output directories.
Task: Append a new section to `.gitignore` covering the directories a bundled build will create: `dist`, `dist-ssr`, `publish`, `preview-root` and the Vite cache directory `.vite`. Keep the existing five entries and their comment untouched, and add a short comment heading for the new group. Also add `*.local`, which is the convention Vite's own template gitignore uses for local-only files.
Constraints: Do not remove or reorder any existing `.gitignore` entry. Do not modify `index.html`, `AGENTS.md`, `package.json` or any workflow file. Do not change the three `localStorage` key names or the persisted data shape; this prompt touches `.gitignore` only. Do not add editor-specific or OS-specific entries such as `.DS_Store` or `.idea` in this change; keep the diff to build output only.
Acceptance criteria: `.gitignore` still contains all five original entries in their original order. It additionally contains `dist`, `dist-ssr`, `publish`, `preview-root`, `.vite` and `*.local`. Creating an empty directory named `dist` with a file inside it leaves git status `--porcelain` showing nothing for that path. The same holds for `publish` and `preview-root`. No tracked file is removed from the index by this change.
Verification: Run `git check-ignore -v dist/index.html publish/index.html preview-root/x.vite/deps foo.local` and confirm every path is reported as ignored by `.gitignore`. Then run `git status --porcelain` and confirm the only change is `.gitignore` itself. Finally run `git ls-files` and confirm the tracked file list is unchanged apart from nothing being added.

PROMPT QW-4: Stop the favicon 404 by referencing an icon at the project path
Context: Repo lesson-planner, single deployed file `index.html` served from the subpath `https://denyslystopadskyy.github.io/lesson-planner/`. Verified with `grep`: `index.html` contains zero link elements and no base element, so the browser falls back to requesting `/favicon.ico` from the domain root. RP-01 recorded this as defect D23, a console error on every single page load, and confirmed that both the domain root and the project path return 404. Measured on 2026-08-20: `https://denyslystopadskyy.github.io/` returns 404, so nothing at the domain root can ever satisfy that request. This is the same absolute-path-versus-subpath failure class as a misconfigured bundler base path.
Task: Add a small icon file to the repository root, named `favicon.svg`, containing a simple flat SVG icon consistent with the app's existing accent colour, which is defined as the CSS custom property `--accent` with value `#4caf50` at `index.html` line 8. Then add a single icon link element inside the head of `index.html`, after the title element on line 6, whose href is the RELATIVE path `favicon.svg` with no leading slash, so that it resolves against the project subpath rather than the domain root. Use type `image/svg+xml`.
Constraints: The href must be relative. A leading slash would request the domain root and reproduce the bug. Do not add a base element. Do not change the existing meta charset, meta viewport or title elements. Do not touch the style block, the script block, or any application logic. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Do not add any external URL, CDN reference or font: the app currently has zero third-party requests and that property must be preserved. Do not add an apple-touch-icon, a web app manifest or a service worker; those are out of scope.
Acceptance criteria: `favicon.svg` exists at the repository root and is under 4 KB. `index.html`

gains exactly one new line inside head, and `grep` for a double-quoted slash immediately after href in that line returns no match. Loading the page from a subpath such as `http://localhost:4173/lesson-planner/` produces a network request for `favicon.svg` under that subpath, not under the server root, and the browser console shows zero errors. The page still renders identically and the file remains a single self-contained HTML document with exactly one script element and one style element.

Verification: Serve the repository so the app is reachable at a subpath, load it with DevTools open, and confirm the Network panel shows `favicon.svg` requested from the subpath with status 200 and the Console panel shows no entries. Run `grep -c "<script" index.html` and `grep -c "<style" index.html` and confirm each returns 1. Confirm the favicon appears in the browser tab.